Quiz 2: Sentiment Analysis

- **Deliverable:** `submission.py`

Goal (what you'll build)

Implement a binary sentiment classifier for short movie-style reviews using a linear model trained with hinge loss via stochastic gradient descent (SGD). You'll write compact, readable code that converts text to features, trains the model, and evaluates error.

What you must implement (in `submission.py`)

Implement the following functions exactly as specified (signatures and behavior matter for grading):

1. `extractWordFeatures(review: str) -> Dict[str, int]`

- Split on whitespace (case-sensitive).
- Return a dictionary mapping each word to its count.

2. `learnPredictor(trainExamples, devExamples, featureExtractor, numIters=20, eta=0.1) -> Dict[str, float]`

- Train a **linear classifier** with **hinge loss** using **SGD**.
- For each training example `(x, y)` with `y ∈ {+1, -1}`:
  - Compute `margin = y * (w · φ(x))`.
  - If `margin < 1`, update **weights**: `w ← w + η * y * φ(x)` (otherwise no update).
- After each epoch (one pass over the training set), compute and **print** training and dev error rates (fraction incorrect). Printing is required for basic sanity checks.

3. `makeCharacterExtractor(n: int)` → returns `extract(review) -> Dict[str, int]`

- Remove whitespace (spaces and tabs) from the review, then extract all **character n-grams** of length `n`.
- Return a dictionary mapping each n-gram to its count.

4. `generateDataset(n_train: int = 800, n_dev: int = 200, seed: int = 1)`

- Provide a small **synthetic dataset generator** (already stubbed in most skeletons).
- We use labels in `{+1, -1}` and include basic negation patterns like “not good”, “not bad”.

> Note: You may add small helper functions, but do not change the required function signatures above.

Model you must use

- **Linear classifier** with score $s(x) = w \cdot \varphi(x)$
- **Prediction:** $\hat{y} = \text{sign}(s(x))$ (ties → `+1`)
- **Loss:** Hinge loss $L(x,y,w) = \max(0, 1 - y\,(w \cdot \varphi(x)))$
- **Training: SGD** with the update $w \leftarrow w + \eta\, y\, \varphi(x)$ when $y(w \cdot \varphi(x)) < 1$
- **No backpropagation / no neural networks** in this assignment.
- **No external libraries** (e.g., **no** scikit-learn). Use only standard Python. Numpy allowed.

Features to implement & use

1. Word features (default)

- Case-sensitive, whitespace tokenization.
- Example: `"good plot not bad" → {"good":1, "plot":1, "not":1, "bad":1}`

2. Character n-gram features (experiment)

- Remove spaces/tabs, extract all substrings of fixed length `n`.

- Example (`n=3`): `"not good" → "notgood" → {"not":1, "otg":1, "tgo":1, "goo":1, "ood":1}`

You should train/evaluate with both word features and at least one character n-gram size (e.g., `n ∈ {3,4,5}`) and compare dev error.

*An **n-gram** is a contiguous sequence of n items (characters, words, etc.) from a given sample of text. For example:*

- *A 3-gram (trigram) of characters in "hello" would be: "hel", "ell", "llo"*

N-grams capture local patterns and are useful for text classification, as they reflect language structure without requiring deep linguistic analysis.

Data

- The assignment is self-contained using a synthetic dataset (generated in your code) for train/test, so you do not need to download anything.
- You can experiment with real data (optional) → **SST-2** (Stanford Sentiment Treebank, binary subset)

Running & expected output

- Your code should be runnable with a simple Python 3
- After each epoch, print lines like:
  - `[epoch 3] train err=0.025 test err=0.28`
- Keep your file self-contained: if you need tiny helpers (e.g., a `dotProduct`), include them in your file.
- **Readable code**: clear variable names, small functions, comments where helpful.

Grading breakdown (typical)

- **Correct feature extractors** (word & char n-gram): 30%
- **SGD hinge-loss learner** (correct update & behavior): 50%
- **Epoch printing & evaluation** (error computation, no crashes): 15%

- **Style/clarity**: 5%

Tips

- Shuffle training examples each epoch.
- Start with `eta = 0.1` and `numIters ≈ 10-20`.
- If training error stalls, try modest tweaks to `eta` or epochs.
- Implement character n-grams cleanly so you can swap feature extractors with one line.
- Write your own code. Discussing ideas is fine; **do not share final code**. Plagiarism detection may be used.

Deliverable checklist

☐ `submission.py` contains: word features, character n-gram extractor, hinge-loss SGD learner, dataset generator.

☐ Prints epoch train/test error.